

Underwriter Class Cheat Sheet

 Underwriter(*, name='Rory', databases=None, update=cfg, log2=cfg, debug=False)^[1]

The Underwriter manages the creation of Aggregate, Severity, Portfolio, BivariateAggregate, PnL, and Distortion objects, maintains a base of named DecL declarations (Recipe records keyed by (kind, name)), and provides the interface to the Lark-based DecL parser. The convenience method  build^[2] is the primary entry point; uw(program) delegates to it. The following tables show all  methods, and fields or properties (used interchangeably). Comments elucidate the meaning of more obscure entries.

1. Specification & creation

name identifying name for this Underwriter instance
databases the load request: name / path / glob, or 'default', 'user', 'all'; a bare Underwriter() loads nothing
update if true, constructed objects are updated (probabilities computed); defaults to config
log2 default log₂ of the number of buckets used to discretize; defaults to config
debug if true, print parser debug messages

2. Update

 build^[2] (primary entry point: parse a single-output DecL program and return the constructed object),
 build_many (parse a multi-output program, construct each, smart-update; returns list[Recipe]), recipes (the recipe base as a (kind, name)-indexed DataFrame, lazy-loaded),  recipe (the one canonical lookup: one Recipe by name; uw[name] and uw[kind, name] delegate to it)^[3],  load (resolve a request and read matching .agg files into the recipe base),
 add_recipe (add or overwrite one parsed declaration),  to_agg (export a selection of recipes to a .agg file; re-loads cleanly),  reload (reset to as-created state and re-read the configured request), lexer, parser (the DecL lexer / parser objects)

3. Moments

None

4. Statistical functions

None

5. Validation

 interpret_file (parse every DecL program in a .agg / .csv file and return per-line error info; collects all errors rather than aborting — validate a file before installing it), test_suite_file (path to the bundled test suite .agg, or None)^[4]

6. Output dataframes

recipes (the only frame: one row per stored declaration; see Update)

7. Reinsurance

None

8. Visualization

None (though  discover(plot=True) plots every match)

9. Risk and pricing

None

10. Approximations

None

11. Meta

 discover (search the recipe base by regex, tags, and optional kind; optionally describe, plot, or return the objects — the directory view),  help (help on methods and properties matching regex),
 available_databases (list .agg files on the search path: cwd / user / default),  resolve_databases (dry run: preview the files a request would load),
 show_settings (print every resolved setting and where its value came from),  reload_settings (re-read the config file / environment),
 write_default_config (write the annotated config template to user_dir), default_dir (bundled .agg databases), user_dir (~/.aggregate), version (aggregate version string)

Notes:

[1]: Arguments are **keyword-only** (leading *): use databases='test_suite', not Underwriter('test_suite'). update and log2 default to the [build] section of config.toml.

[2]: Also exported as from aggregate import build. The module-level build pre-loads build.databases (test_suite) whereas a bare Underwriter() starts empty. build returns one object, build_many a list.

[3]: A Recipe is one declaration record: kind, name, spec, program, source, the trailer (note, tags, hints, doc), the canonical decl re-rendering, and the cookbook sections (problem, solution, discussion, check). Lookup is pure: Recipe.object is None until build runs.

[4]: aggregate/agg/test_suite.agg (categories A–O) is exercised by uv run pytest, not by an Underwriter method.

kind ∈ {sev, agg, port, distortion, bvaagg, pnl, xpnl, expr}; database search path cwd → user_dir → default_dir.

